```
// Programmed by: Jhaztintroy Baluyot
package com.mycompany.baluyotproblem3;

import java.util.Scanner;

class InventoryException extends Exception {
    public InventoryException(String message) { super(message); }
}

class Product {
    private final int id;
    private final String name;
    private int quantity;

    public Product(int id, String name, int quantity) throws InventoryException {
        if (name == null || name.trim().isEmpty())
            throw new InventoryException("Name cannot be empty.");
        if (quantity < 0)
            throw new InventoryException("Quantity cannot be negative.");
        this.id = id;
        this.name = name.trim();
        this.quantity = quantity;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public int getQuantity() { return quantity; }

    public void restock(int amount) throws InventoryException {
        if (amount <= 0) throw new InventoryException("Amount must be positive.");
        quantity += amount;
    }

    public void sell(int amount) throws InventoryException {
        if (amount <= 0) throw new InventoryException("Amount must be positive.");
        if (amount > quantity) throw new InventoryException("Insufficient stock.");
        quantity -= amount;
    }

    @Override
    public String toString() {
        return id + " " + name + " Quantity " + quantity;
    }
}

class Inventory {
```

```
    private static final int CAPACITY = 4;
    private final Product[] products = new Product[CAPACITY];
    private int count = 0;

    public Product findById(int id) {
        for (int i = 0; i < count; i++)
            if (products[i].getId() == id) return products[i];
        return null;
    }

    private boolean nameExists(String name) {
        for (int i = 0; i < count; i++)
            if (products[i].getName().equalsIgnoreCase(name.trim())) return true;
        return false;
    }

    public void add(int id, String name, int quantity) throws InventoryException {
        if (quantity < 0) throw new InventoryException("Quantity cannot be
negative.");
        if (findById(id) != null) throw new InventoryException("Duplicate product
ID.");
        if (nameExists(name)) throw new InventoryException("Duplicate product
name.");
        if (count >= CAPACITY) throw new InventoryException("Inventory is full.");
        products[count++] = new Product(id, name, quantity);
    }

    public Product restock(int id, int amount) throws InventoryException {
        Product p = findOrThrow(id);
        p.restock(amount);
        return p;
    }

    public Product sell(int id, int amount) throws InventoryException {
        Product p = findOrThrow(id);
        p.sell(amount);
        return p;
    }

    public Product[] getProducts() {
        Product[] copy = new Product[count];
        for (int i = 0; i < count; i++) copy[i] = products[i];
        return copy;
    }

    private Product findOrThrow(int id) throws InventoryException {
        Product p = findById(id);
        if (p == null) throw new InventoryException("Product not found.");
        return p;
    }
```

```
}

public class Baluyotproblem3 {
    private static final Scanner in = new Scanner(System.in);

    private static int readInt(String prompt) throws InventoryException {
        System.out.print(prompt);
        try {
            return Integer.parseInt(in.nextLine().trim());
        } catch (NumberFormatException e) {
            throw new InventoryException("Enter a valid number.");
        }
    }

    public static void main(String[] args) {
        Inventory inventory = new Inventory();
        boolean running = true;
        while (running) {
            System.out.println("\nMenu: 1 Add | 2 Restock | 3 Sell | 4 List | 0
Exit");
            try {
                int choice = readInt("Choice: ");
                switch (choice) {
                    case 1: {
                        int id = readInt("ID: ");
                        System.out.print("Name: ");
                        String name = in.nextLine();
                        int qty = readInt("Starting quantity: ");
                        inventory.add(id, name, qty);
                        System.out.println("Added");
                        break;
                    }
                    case 2: {
                        int id = readInt("ID: ");
                        int amount = readInt("Amount: ");
                        System.out.println("Quantity " + inventory.restock(id,
amount).getQuantity());
                        break;
                    }
                    case 3: {
                        int id = readInt("ID: ");
                        int amount = readInt("Amount: ");
                        System.out.println("Quantity " + inventory.sell(id,
amount).getQuantity());
                        break;
                    }
                    case 4: {
                        Product[] all = inventory.getProducts();
                        if (all.length == 0) System.out.println("No products.");
                        for (Product p : all) System.out.println(p);
```

```
                        break;
                    }
                    case 0:
                        running = false;
                        break;
                    default:
                        throw new InventoryException("Invalid menu choice.");
                }
            } catch (InventoryException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}
```